

Applications of the Bayesian Network Interpretation of Transformers

Greg Coppola

2026

Abstract

The companion paper [2] proves that a sigmoid transformer can exactly realize belief propagation under a constructive weight mapping. This paper develops the applications and implications of that result. We organize them across six domains: theoretical foundations (why LLMs work, scaling laws, formal reasoning), grounding and hallucination, agents and planning, mechanistic interpretability, training objectives, and efficiency. Each application is marked by its epistemic status — formal consequence, mechanistic interpretation, empirical hypothesis, architectural proposal, or speculative extension — so that the reader can calibrate confidence accordingly. The paper concludes with an explicit account of what the theory does not prove and a research roadmap for the open problems.

Contents

I	Introduction and Framework	3
1	Introduction	3
II	Theoretical Consequences	4
2	Why LLMs Work: A Theoretical Foundation	4
3	Scaling Laws Have a Mechanistic Interpretation	5
4	Formal Reasoning as a Special Case	6
III	Grounding and Hallucination	7
5	Hallucination Has a Formal Definition	7
6	The Grounding Gap	8
IV	Agents and Planning	8

7	Agents and the BP Vocabulary	9
8	Planning as Inference	9
9	Multi-Agent Systems and Distributed Inference	10
V	Interpretability	11
10	Interpretability as Hypothesis Testing	11
11	SAE Features as Factor Graph Nodes	12
VI	Training and Efficiency	13
12	Training Objectives Through the BP Lens	13
13	Sparse Inference and Pruning	14
VII	Limitations and Open Problems	14
14	What This Does Not Prove	14
15	Research Roadmap	15

Part I

Introduction and Framework

1 Introduction

The companion paper [2] proves that a sigmoid transformer can exactly realize belief propagation (BP) under a constructive weight mapping. This is not a loose analogy or a high-level functional similarity: the forward pass implements the BP update equations directly, with residual streams as belief states, attention as evidence gathering, and feed-forward networks as belief updates.

The present paper asks: what follows from taking that result seriously? The applications range from theoretical (a mechanistic account of why LLMs work) to practical (grounding strategies, interpretability predictions, agent design) to open research directions (calibrated uncertainty, factor graph recovery, grounded planning systems).

Levels of Analysis

The applications in this paper operate at several distinct levels that should not be conflated:

Level	Description	Example
Exact constructive	Formally verified theorem	Sigmoid transformer realizes exact BP
Direct consequence	Follows from the theorem	Grounded BP has no hallucination
Mechanistic interpretation	Component-level account	Attention = evidence gathering
Empirical hypothesis	Testable prediction	AND heads have rank-1 Q/K circuits
Architectural proposal	Design suggestion	Grounded transformer + QBBN hybrid
Speculative extension	Interpretive extrapolation	Planning as MDP inference

Each section marks the epistemic status of its claims explicitly.

Map of Literatures

Five independent research programs have converged on related pictures of transformer computation. The BP result provides a common language for organizing their findings:

Literature	Main Idea	BP Connection
Mechanistic interpretability	Circuits and routing heads	Message passing structure
In-context learning	Bayesian updating from examples	Posterior inference per forward pass
Graph neural networks	Node message passing	Explicit graph BP
Sparse autoencoders	Monosemantic semantic features	Latent factor graph nodes
RAG / tool use	External evidence injection	Conditioning on observed evidence

What This Paper Is Not Claiming

The formal theorem is about sigmoid transformers under a specific constructive weight mapping. Production transformers use SwiGLU or GeLU, not sigmoid. The theorem establishes that the architecture *can* implement exact BP; whether any particular trained model does is an empirical question. The applications that follow are interpretations, proposals, and hypotheses motivated by the result — not derived consequences of it, except where explicitly noted.

Part II

Theoretical Consequences

2 Why LLMs Work: A Theoretical Foundation

There is currently no widely accepted mechanistic theory of why next-token prediction on text produces general intelligence. The standard story is empirical: scaling laws hold, capabilities emerge, benchmarks improve. Other theoretical programs exist — information-theoretic accounts, compression views, in-context learning theory, grokking — but none has achieved consensus as a mechanistic explanation of what the model is computing.

The BP Interpretation

If transformers implement belief propagation over an implicit factor graph, then next-token prediction can be interpreted as marginal inference in a Bayesian network over language concepts. Under this interpretation, the model behaves as though it is learning a structured probabilistic model of the world and performing inference in it.

This suggests a mechanistic account of what the model is computing: each forward pass propagates beliefs through an implicit factor graph, updating each proposition’s probability given the evidence in the context window. Next-token prediction is the output of this inference process — the most probable next token given the current belief state.

Scaling Through the BP Lens

The scaling dimensions have natural BP interpretations:

Dimension	BP Interpretation
W (sequence length)	Nodes in the factor graph
L (layers)	Rounds of belief propagation
H (attention heads)	AND patterns per round
D_{ff} (FFN hidden units)	OR patterns per round
D_{model} (embedding)	Belief state capacity

Scaling may improve performance because it expands inference capacity — more rounds, richer potentials, more simultaneous concepts. This is qualitatively consistent with the BP interpretation, though the specific power-law exponents are not derived from BP theory.

What This Does Not Explain

The BP framework suggests a structure for what the model computes. It does not explain why the training distribution (internet text) produces a useful factor graph, nor does it derive the power-law scaling exponents from first principles. These remain open questions.

Epistemic Status

Claim	Status
Sigmoid transformers can implement BP	Formal theorem
Trained LLMs implement BP	Empirical hypothesis
Next-token prediction fits a Bayes net	Mechanistic interpretation
Scaling expands BP inference capacity	Mechanistic interpretation
BP explains power-law scaling exponents	Not established

3 Scaling Laws Have a Mechanistic Interpretation

Kaplan et al. and subsequent work established that loss scales predictably as a power law with compute, parameters, and data [4]. This is an empirical regularity. No principled derivation of the exponents has been established, though several theoretical programs have attempted partial accounts.

The BP Interpretation

Under the BP interpretation, scaling adds inference capacity in well-defined ways: more layers add rounds of BP, more D_{ff} adds OR gate capacity, more D_{model} adds superposition capacity, and more W adds factor graph nodes. Each of these is a smooth, monotone increase in a well-defined quantity. Power-law scaling in loss is qualitatively consistent with a system where each additional unit of capacity provides diminishing but consistent marginal improvement in inference quality over a fixed distribution.

The AND Head Invariant

Empirically, boolean-AND head count stays roughly fixed within a model family while hub and OR capacity scales. This is consistent with the BP interpretation: the routing primitive is compact and does not need to scale, while inference capacity does. This evidence is preliminary — three model families — and should not be overstated.

Epistemic Status

Claim	Status
Scaling dimensions have BP interpretations	Mechanistic interpretation
More layers enable deeper reasoning chains	Mechanistic interpretation
AND head invariance is consistent with BP	Empirical observation (preliminary)
BP explains power-law exponents	Not established

4 Formal Reasoning as a Special Case

Classical first-order logic and probabilistic graphical models are usually presented as separate formalisms. The QBBN [1] unifies these: it is a probabilistic graphical model over propositions with AND/OR boolean structure, proven consistent and complete with respect to the first-order calculus. Logical deduction is the special case where the OR gate weights are deterministic; statistical inference is the general case where the weights are soft and learned from data.

The Controlled/Learned Spectrum

This suggests a spectrum between controlled and learned reasoning:

- **Fully controlled:** OR gate weights are fixed to be deterministic. The system implements exact logical deduction. This is the QBBN in symbolic mode.
- **Fully learned:** OR gate weights are learned from data. The system implements soft probabilistic inference. This is the standard LLM.
- **Hybrid:** some weights are fixed (known logical rules) and some are learned (uncertain statistical relationships). This is the target architecture for systems combining flexibility with formal reliability.

The transformer’s BP structure is compatible with all three modes. Current training produces the fully learned mode.

The Completeness Connection — and Its Limits

The QBBN paper proves consistency and completeness with respect to the first-order calculus. This is a result about *representational expressivity*, not about practical inference capability. It says the QBBN can in principle express any first-order fact — not that a transformer will reliably compute it, nor that inference will be tractable, nor that the right weights will be learned from data.

Concretely: a transformer with grounded BP weights is not thereby guaranteed to do mathematics reliably. Mathematical reasoning requires not just the right representational structure but also the right weights, sufficient layers for the required inference depth, and a training procedure that produces calibrated beliefs. The completeness result says the architecture is not the bottleneck. It does not say the other bottlenecks are easy.

Fast vs. Slow Reasoning

The QBBN analysis connects to Kahneman’s fast/slow distinction [3]. Fast reasoning is forward inference: causes propagate to effects in one pass of BP. Slow reasoning requires reasoning by cases — exploring multiple hypotheses, maintaining multiple belief states, backtracking. The number of transformer layers bounds the depth of fast reasoning; slow reasoning requires iteration or explicit search outside the model.

Epistemic Status

Claim	Status
QBBN is consistent and complete w.r.t. first-order logic	Formal theorem
Transformers are compatible with formal reasoning	Direct consequence
Trained transformers reliably do formal reasoning	Not established
Completeness implies practical mathematical competence	Explicitly not claimed
Hybrid architecture is achievable	Architectural proposal

Part III

Grounding and Hallucination

5 Hallucination Has a Formal Definition

Hallucination is typically described informally: the model says something false or unsupported. This is a behavioral description, not a mechanistic one. It gives no account of why hallucination happens or what would prevent it.

The BP Definition

The BP framework provides a formal definition: a model hallucinates when its output beliefs diverge from the correct posterior given the evidence. For a grounded transformer with a declared ontology and verifier, this divergence is measurable:

$$b(j) \neq P_{\text{true}}(j),$$

where P_{true} is the correct posterior induced by the knowledge base and observed evidence.

For an ungrounded transformer, the comparison is not well-defined because there is no formally declared ground truth. Hallucination in ungrounded models is not a numerical error in the inference procedure — it is a consequence of operating without a fully specified and verifiable grounding structure.

What Grounding Requires

A fully grounded transformer would need: a declared concept space, a declared relational structure, a grounding procedure mapping natural language to the declared system, and a verification mechanism. Retrieval- augmented generation is a partial form of grounding: it temporarily constrains the inference space but does not provide persistent ontological structure or a verifier.

Epistemic Status

Claim	Status
Grounded BP systems have a formal hallucination definition	Direct consequence
Ungrounded models lack a formal correctness criterion	Direct consequence
Hallucination requires grounding to formally eliminate	Mechanistic interpretation
Scaling alone cannot eliminate hallucination	Mechanistic interpretation

6 The Grounding Gap

There are two distinct kinds of reliability a language model can have. *Empirical reliability* means the model produces correct outputs most of the time on a given distribution, as measured by benchmarks or human evaluation. *Formal reliability* means the model’s outputs are guaranteed correct relative to a specified knowledge base and ontology, by a verifiable proof.

Current LLMs have empirical reliability. Grounded BP systems — transformers operating over a declared factor graph with a verifier — have formal reliability in restricted settings. The gap between these two is the grounding gap.

Why Scaling Does Not Close It

Scaling improves empirical reliability monotonically. But it does not create a declared ontology, a finite concept space, or a verifier. A 405B parameter model with no grounding structure cannot provide formal correctness guarantees, regardless of its benchmark performance.

The grounding gap is not a quantitative gap that more parameters will close. It is a structural gap between two different kinds of system.

What Would Close It

Closing the grounding gap requires building systems that combine neural inference (the transformer’s BP computation) with symbolic grounding (a declared ontology and verifier). This is the QBBN program. Whether it can be scaled to general-purpose language understanding is the central open problem.

Epistemic Status

Claim	Status
Two kinds of reliability are distinct	Conceptual distinction
Grounded BP systems have formal reliability	Direct consequence
Scaling does not close the grounding gap	Mechanistic interpretation
QBBN hybrid closes the gap	Architectural proposal

Part IV

Agents and Planning

7 Agents and the BP Vocabulary

An LLM agent is typically described as: perception $\rightarrow$ reasoning $\rightarrow$ action, iterated. This description is functional but not formal. The BP framework provides a formal vocabulary for describing each component — not a proof that agents are Bayesian networks in every mechanistic detail, but a claim that the BP language maps naturally onto agent components and may provide useful formal tools.

Under this interpretation: perception can be described as evidence injection; reasoning as BP inference; action as decision inference over the posterior; memory as persistent factor graph state; and tool calls as factor graph queries. Whether these correspondences are exact or approximate depends on how grounded the underlying factor graph is. For an ungrounded LLM, they are interpretive. For a grounded transformer operating over a declared QBBN, they become formal.

The QBBN as the Logic Layer

The structured reasoning connecting perception to action can be formalized as a QBBN with AND/OR factor structure. The LLM provides neural inference capacity; the QBBN provides logical structure that makes reasoning compositional and verifiable. This suggests a hybrid architecture where every component is described in the same probabilistic language. Whether this can be built and scaled is an open research question.

Epistemic Status

Claim	Status
BP vocabulary maps onto agent components	Mechanistic interpretation
LLM reasoning is BP inference	Empirical hypothesis
Grounded agent loop has formal BP account	Architectural proposal
Formal verification of agent steps is possible	Speculative extension

8 Planning as Inference

Planning is the problem of selecting actions to achieve goals. The standard formal model is the Markov Decision Process (MDP): states S , actions A , transition probabilities $P(s'|s, a)$, reward $R(s, a)$, and discount γ . MDPs can be represented as dynamic Bayesian networks. Policy evaluation is inference; policy improvement is optimization over the inferred value function.

The Bellman Equation as Message Passing

The Bellman equation

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

can be interpreted as a message passing update: the value at state s is computed from the values of its neighbors, weighted by transition probabilities. Under this interpretation, a transformer running BP over an MDP factor graph computes value estimates iteratively across layers, with each layer corresponding to one step of policy evaluation. This is a mechanistic interpretation, not a proven equivalence for arbitrary trained transformers.

Chain-of-Thought as Extended Inference

When an LLM reasons about future states in a chain-of-thought, it is plausibly approximating this process. One interpretation of why chain-of-thought helps planning: it provides more effective rounds of inference by externalizing intermediate estimates into the context window. This is consistent with the BP interpretation but not uniquely predicted by it.

Grounded Planning

A grounded planning system would combine a formally declared state space (QBBN propositions), learned or specified transition probabilities (OR gate weights), a BP inference engine (the transformer), and a verifier. Whether this architecture can be built and scaled is an open research question.

Epistemic Status

Claim	Status
MDPs can be represented as dynamic Bayes nets	Established (standard formalism)
Bellman equation resembles BP message passing	Mechanistic interpretation
Chain-of-thought provides more inference rounds	Empirical hypothesis
Ungrounded LLMs fail at long-horizon planning	Empirical observation
Grounded transformer enables verified planning	Architectural proposal

9 Multi-Agent Systems and Distributed Inference

If each agent is performing BP over its local factor graph, then a multi-agent system can be interpreted as a distributed inference computation over a larger global factor graph. This resembles loopy belief propagation on a graph too large to fit in a single model.

This is an interpretive framework, not a proven equivalence. Real LLM agents are not calibrated BP nodes. Communication between agents is lossy text, not exact belief messages. The convergence guarantees from formal loopy BP do not transfer directly to multi-agent LLM systems.

What the BP Lens Suggests

Even without exact equivalence, the distributed BP framing suggests useful design principles: agent boundaries may work better when aligned with factor graph structure; message formats carrying structured uncertainty may coordinate better than raw text; convergence criteria inspired by BP belief stability could inform stopping rules for multi-agent deliberation.

What Would Be Required for a Formal Account

For the distributed BP interpretation to become a formal account, agents would need to maintain calibrated beliefs over a shared declared factor graph, pass approximately correct belief messages, update beliefs according to BP rules, and operate over a graph structure where loopy BP converges. None of these conditions hold for current LLM multi-agent systems.

Epistemic Status

Claim	Status
Multi-agent systems resemble distributed BP	Mechanistic interpretation
BP lens suggests useful design principles	Architectural proposal
Current LLM agents converge to correct posteriors	Not supported
BP-inspired coordination protocols may help	Speculative hypothesis

Part V

Interpretability

10 Interpretability as Hypothesis Testing

Mechanistic interpretability is currently largely exploratory. The BP framework changes this: it generates specific, testable predictions about what trained transformers should look like at the weight level.

Specific Predictions

- Boolean-AND heads should have approximately rank-1 Q/K circuits consistent with `projectDim`-style matching.
- The dominant singular vector of the Q/K circuit should correspond to a monosemantic SAE feature.
- The V circuit should show `crossProject`-style routing from a belief-encoding dimension to a scratch-slot dimension.
- W_1 rows of the FFN should cluster by input SAE feature.
- W_2 columns should correspond to interpretable belief updates.

- The relationship between input and output SAE features for each FFN unit should be interpretable as a conditional probability table entry.

These are specific structural claims that can be confirmed or falsified by inspecting the weights of a trained model with a good SAE.

What a Negative Result Would Mean

If boolean-AND heads do not show rank-1 Q/K structure, that would falsify the constructive BP account for those heads — not just complicate it. The BP framework is a genuine scientific hypothesis because it makes predictions that could be wrong.

The Closing Experiment

Take a model with a trained SAE. Identify boolean-AND heads by behavior. Project the Q/K matrix onto the SAE feature basis. Check if the dominant component corresponds to a monosemantic feature. Check if the V circuit routes from a belief dimension to a scratch slot. Check if input/output SAE feature pairs for active FFN units are interpretable as conditional relationships. If this succeeds, the implicit factor graph is legible.

Epistemic Status

Claim	Status
BP predicts rank-1 Q/K structure in AND heads	Empirical hypothesis
SAE features correspond to factor graph nodes	Empirical hypothesis
Factor graph is legible from weights + SAE	Empirical hypothesis
Negative SAE result would falsify BP account	Direct consequence

11 SAE Features as Factor Graph Nodes

FFN hidden units are not the right unit for understanding the implicit factor graph. A single hidden unit may participate in multiple factor relationships via superposition, and a single factor relationship may require multiple hidden units to implement accurately with non-sigmoid activations.

The right unit is the SAE feature. Each monosemantic SAE feature — one that activates for a single coherent concept — corresponds to one node in the implicit factor graph. The feature direction in residual stream space is where the belief value for that concept lives.

The Factor Graph is Readable in Principle

Given a complete SAE, the full implicit factor graph is readable in principle: factor graph nodes are monosemantic SAE features; edges are W_1 rows that respond to pairs of input features whose W_2 columns write to output features; factor potentials are the learned weights connecting input to output features.

Current SAEs cover a small fraction of the residual stream’s representational capacity. As coverage improves, the factor graph becomes more legible. The BP result gives interpretability research a specific target: complete the SAE, then read the graph.

Epistemic Status

Claim	Status
SAE features correspond to factor graph nodes	Empirical hypothesis
Factor graph edges are readable from W1/W2	Empirical hypothesis
Complete SAE would make factor graph legible	Architectural proposal
Golden Gate Claude demonstrates evidence injection	Empirical observation

Part VI

Training and Efficiency

12 Training Objectives Through the BP Lens

Next-token prediction optimizes cross-entropy loss on the training distribution. This is equivalent to maximum likelihood estimation of the implicit factor graph parameters. The bayes-learner experiment confirms that gradient descent finds approximately correct BP weights from this objective (val MAE 0.000752).

Alternative Objectives

The BP framework suggests objectives that would produce better-calibrated Bayes nets:

- **Calibration objectives:** penalize overconfident or underconfident posteriors directly, not just cross-entropy.
- **Grounding objectives:** reward outputs that are verifiable against a declared knowledge base.
- **BP consistency objectives:** penalize violations of the BP fixed-point equations across layers. If beliefs at layer $l + 1$ are inconsistent with what BP would predict given beliefs at layer l , penalize that divergence.

The Uniqueness Theorem as a Training Target

The companion paper proves a uniqueness result: exact posteriors uniquely force BP weights. This means there is a well-defined target for training — the weights that implement exact BP — and the question is how well current training objectives approach it.

Epistemic Status

Claim	Status
Gradient descent finds approximate BP weights	Empirical observation
Calibration objectives would improve reliability	Architectural proposal
BP consistency training would enforce layer-wise calibration	Architectural proposal
Uniqueness theorem gives a well-defined training target	Direct consequence

13 Sparse Inference and Pruning

The implicit factor graph is sparse: each node has only a few neighbors, and most conditional relationships are zero or near-zero. If the sparse factor graph structure can be identified — which SAE features matter for a given task, which AND connections are active, which OR gates are relevant — the rest can be skipped. This is structured pruning with a theoretical justification.

Task-Specific Factor Graphs

Different tasks activate different subgraphs of the implicit factor graph. This is the theoretical foundation for mixture-of-experts and sparse attention: they are approximations to task-specific factor graph routing, and the BP framework gives a principled account of what they are approximating.

Distillation as Factor Graph Compression

Knowledge distillation can be understood as factor graph compression: finding a smaller factor graph that approximates the posteriors of the larger one. The BP framework gives a principled objective for distillation: minimize KL divergence between the posteriors of the teacher and student factor graphs, not just between their output distributions.

Epistemic Status

Claim	Status
Implicit factor graph is sparse	Mechanistic interpretation
Task-specific subgraph activation motivates sparse computation	Mechanistic interpretation
MoE approximates task-specific factor graph routing	Mechanistic interpretation
Distillation as factor graph compression	Architectural proposal

Part VII

Limitations and Open Problems

14 What This Does Not Prove

The result that sigmoid transformers can exactly realize belief propagation is formal and verified. The applications that follow range from well-established to speculative. This section states clearly what the theory does not prove.

The Theorem Is About Sigmoid Transformers

Production transformers use SwiGLU or GeLU, not sigmoid activation. The exact BP equivalence holds for sigmoid transformers under the constructive weight mapping. For production transformers, the architecture is compatible with BP-like computation and gradient

descent finds approximately correct BP solutions empirically, but exact equivalence is not established.

Attention Is Not Literally Logical Conjunction

The AND/OR framing is mechanistically motivated but is an interpretation, not a theorem about arbitrary trained attention heads. Roughly 90% of heads across studied models fall into the boolean-AND, continuous-OR, and hub categories. The remaining ~10% are genuinely uncharacterized.

The Factor Graph Is Not Directly Observable

The implicit factor graph is encoded in learned weights, not explicitly declared. Current SAEs cover a small fraction of the residual stream’s representational capacity. Factor graph edges have not been systematically recovered. The claim that SAE features correspond to factor graph nodes is an empirical hypothesis with supporting evidence, not a proven theorem.

What Would Falsify the Core Interpretation

The BP interpretation would be significantly weakened if: boolean-AND heads do not exhibit rank-1 Q/K structure under SAE analysis; SAE features fail to organize into stable dependency structure; FFN updates are not locally interpretable as belief updates; deeper layers do not improve iterative reasoning behavior; or gradient descent does not find BP-like solutions on clean synthetic tasks.

15 Research Roadmap

The BP interpretation of transformers is an active empirical program, not a finished doctrine. The open research directions divide into three tiers by tractability.

Tier 1 — Most Directly Testable

Recover factor graphs from SAEs. For each boolean-AND head, project the Q/K matrix onto the SAE feature basis and check if the dominant component corresponds to a monosemantic feature. Check if the V circuit routes from a belief dimension to a scratch slot.

Test layer-wise belief convergence. Train a tiny transformer on a synthetic Bayes net with known ground-truth posteriors. Measure how closely the residual stream at each layer matches the BP belief after that many rounds of message passing.

Measure calibration by layer depth. Compare output calibration of models with different depths on multi-step reasoning tasks. If deeper models are better calibrated on longer chains, that supports the layers-as-BP-rounds account.

Tier 2 — Requires New Infrastructure

Complete SAE coverage of a production model. Build and benchmark a grounded transformer + QBBN hybrid. Develop a calibration benchmark for BP consistency that measures layer-wise belief stability.

Tier 3 — Longer-Term

Learning grounded factor graphs from text via expectation maximization. Formal account of in-context learning as BP. Multi-agent BP convergence conditions.

What Would Most Advance the Program

In order of expected impact: (1) the SAE factor graph recovery experiment, which closes the loop between the formal theorem and trained models; (2) layer-wise convergence measurement, which tests the layers-as-rounds claim directly; and (3) the grounded hybrid system benchmark, which demonstrates the practical value of grounding.

References

- [1] Greg Coppola. The quantified Boolean Bayesian network: Theory and experiments with a logical graphical model. *arXiv preprint arXiv:2402.06557*, 2024.
- [2] Greg Coppola. Transformers are Bayesian networks. *arXiv preprint arXiv:2603.17063*, 2026.
- [3] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [4] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.